

HDOC Day-17

Write a menu – driven program using switch-case to perform digit – based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and a simple function call. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write , compile , execute , and test the C program against the prepared use cases.

- Generate the largest possible number using its digit.
- Generate the smallest possible number using its digit.
- Swap its first and last digit.
- Check whether it is a palindrome number.

Algorithm:

- **Step 1: Start**
 - Begin the program.
- **Step 2: Display Menu**
 - Display the following menu:
 1. Generate largest possible number using digits
 2. Generate smallest possible number using digits
 3. Swap first and last digits
 4. Check whether the number is palindrome
 5. Exit
- **Step 3: Read Choice**
 - Ask the user to enter their choice.
- **Step 4: Use Switch-Case**
 - Use switch-case according to the user's choice.
 - **Case 1:** Call largest() function.
 - **Case 2:** Call smallest() function.
 - **Case 3:** Call swapFirstLast() function.

- **Case 4:** Call palindrome() function.
- **Case 5:** Terminate the program.
- **Default:** Display "Invalid choice".

• Step 5: Repeat

- Repeat the menu until the user selects **5 (Exit)**.

• Step 6: Stop

- End the program.

• Algorithm for largest()

1. Start the largest() function.
2. Read a positive integer n.
3. Extract all digits of n and store them in an array.
4. Sort the digits in **descending order**.
5. Display the digits in the sorted order.
6. The resulting number is the **largest possible number**.
7. Return to the main menu.

- **Example:**

42153 → 54321

• Algorithm for smallest()

1. Start the smallest() function.
2. Read a positive integer n.
3. Extract all digits of n and store them in an array.
4. Sort the digits in **ascending order**.
5. If the first digit is 0, interchange it with the first non-zero digit.
6. Display the digits.
7. The resulting number is the **smallest possible number**.
8. Return to the main menu.

- **Example:**

42153 → 12345

• Algorithm for swapFirstLast()

1. **Start** the swapFirstLast() function.
2. Read a positive integer n.
3. Extract all digits and store them in an array.
4. Identify the first and last digits.
5. Swap the first and last digits.

6. Display the resulting number.
7. Return to the main menu.

- **Example:**

42153 → 32154

-

- **Algorithm for palindrome()**

1. Start the palindrome() function.
2. Read a positive integer n.
3. Store the original number in original.
4. Set reverse = 0.
5. Extract the last digit using $n \% 10$.
6. Add the digit to reverse.
7. Remove the last digit using $n = n / 10$.
8. Repeat Steps 5–7 until n becomes 0.
9. Compare original with reverse.
10. If both are equal, display "**Palindrome number**".
11. Otherwise, display "**Not a palindrome number**".
12. Return to the main menu.

- **Example:**

1221 → Palindrome

1234 → Not a palindrome

- **C Program:**

- `#include <stdio.h>`
-
- `// Function declarations`
- `void largest();`
- `void smallest();`
- `void swapFirstLast();`
- `void palindrome();`
-
- `int main()`
- `{`
- `int choice;`
-
- `do`
- `{`

- printf("\n===== MENU =====\n");
- printf("1. Generate largest possible number\n");
- printf("2. Generate smallest possible number\n");
- printf("3. Swap first and last digits\n");
- printf("4. Check palindrome\n");
- printf("5. Exit\n");
- printf("=====\n");
-
- printf("Enter your choice: ");
- scanf("%d", &choice);
-
- switch(choice)
- {
- case 1:
- largest();
- break;
-
- case 2:
- smallest();
- break;
-
- case 3:
- swapFirstLast();
- break;
-
- case 4:
- palindrome();
- break;
-
- case 5:
- printf("Program terminated.\n");
- break;
-
- default:
- printf("Invalid choice! Please enter 1-5.\n");
- }
-
- } while(choice != 5);
-

- return 0;
- }
-
-
- // Function to generate largest possible number
- void largest()
- {
- int n, digit[20], count = 0;
- int temp, i, j;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- temp = n;
-
- while(temp > 0)
- {
- digit[count] = temp % 10;
- temp = temp / 10;
- count++;
- }
-
- // Sort digits in descending order
- for(i = 0; i < count - 1; i++)
- {
- for(j = i + 1; j < count; j++)
- {
- if(digit[i] < digit[j])
- {
- int t = digit[i];
- digit[i] = digit[j];
- digit[j] = t;
- }
- }
- }
- }
-
- printf("Largest possible number = ");
-
- for(i = 0; i < count; i++)

- {
- printf("%d", digit[i]);
- }
-
- printf("\n");
- }
-
-
- // Function to generate smallest possible number
- void smallest()
- {
- int n, digit[20], count = 0;
- int temp, i, j;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- temp = n;
-
- while(temp > 0)
- {
- digit[count] = temp % 10;
- temp = temp / 10;
- count++;
- }
-
- // Sort digits in ascending order
- for(i = 0; i < count - 1; i++)
- {
- for(j = i + 1; j < count; j++)
- {
- if(digit[i] > digit[j])
- {
- int t = digit[i];
- digit[i] = digit[j];
- digit[j] = t;
- }
- }
- }
- }

-
- // Avoid leading zero
- if(digit[0] == 0)
- {
- for(i = 1; i < count; i++)
- {
- if(digit[i] != 0)
- {
- int t = digit[0];
- digit[0] = digit[i];
- digit[i] = t;
- break;
- }
- }
- }
- }
-
- printf("Smallest possible number = ");
-
- for(i = 0; i < count; i++)
- {
- printf("%d", digit[i]);
- }
-
- printf("\n");
- }
-
-
- // Function to swap first and last digits
- void swapFirstLast()
- {
- int n, digit[20], count = 0;
- int temp, i;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- temp = n;
-
- while(temp > 0)

- {
- digit[count] = temp % 10;
- temp = temp / 10;
- count++;
- }
-
- // Swap first and last digits
- int t = digit[0];
- digit[0] = digit[count - 1];
- digit[count - 1] = t;
-
- printf("Number after swapping first and last digits = ");
-
- // Print in reverse because digits were stored backwards
- for(i = count - 1; i >= 0; i--)
- {
- printf("%d", digit[i]);
- }
-
- printf("\n");
- }
-
-
- // Function to check palindrome
- void palindrome()
- {
- int n, original, reverse = 0, digit;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);
-
- original = n;
-
- while(n > 0)
- {
- digit = n % 10;
- reverse = reverse * 10 + digit;
- n = n / 10;
- }

-
- if(original == reverse)
- {
- printf("%d is a palindrome number.\n", original);
- }
- else
- {
- printf("%d is not a palindrome number.\n", original);
- }
- }

Test Case Table:

Test Case	Choice	Input Number	Expected Output	Actual Output
1	1	42153	Largest possible= 54321	54321
2	1	12345	Largest Possible Number= 54321	54321
3	2	42153	Smallest Possible number= 12345	12345
4	2	50421	Smallest Possible number= 10245	10245
5	3	42153	After swapping first and last digits=32154	32154
6	3	12345	After swapping first and last digits= 52341	52341
7	4	1221	1221 is a palindrome number	1221 is a palindrome

8	4	1234	1234 is a palindrome number	1234 is a palindrome
9	4	121	121 is a palindrome	121 is a palindrome
10	5	-	Program terminates	Program Terminated
11	6	-	Invalid choice message	Invalid choice

Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:

```

—————MENU—————
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
5. Exit

Enter your choice: 1
Enter a positive integer: 42153
Largest possible number = 54321

```

Test Case 2:

```

—————MENU—————
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
5. Exit

Enter your choice: 1
Enter a positive integer: 12345
Largest possible number = 54321

```

Test Case 3:

```
—————MENU—————  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
5. Exit  
  
Enter your choice: 3  
Enter a positive integer: 42153  
Number after swapping first and last digits = 32154
```

Test Case 4:

```
—————MENU—————  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
5. Exit  
  
Enter your choice: 3  
Enter a positive integer: 12345  
Number after swapping first and last digits = 52341
```

Test Case 5:

```
—————MENU—————  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
5. Exit  
  
Enter your choice: 4  
Enter a positive integer: 1221  
1221 is a palindrome number.
```

Test Case 6:

MENU

1. Generate largest possible number
 2. Generate smallest possible number
 3. Swap first and last digits
 4. Check palindrome
 5. Exit
-

Enter your choice: 1234

Invalid choice! Please enter 1-5.

MENU

1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
5. Exit